

RRC-Raptor-V4H

- Porting Guide

V0.0.1

Contents

i. VERSION LIST	4
ii. HASHTAG	5
1. Overview.....	6
2. System Requirements.....	7
3. Setup.....	8
3.1. Host Setup	9
3.2. Host Packages.....	10
3.3. Yocto Project Setup	11
3.4. Source Download	12
3.4.1. Using Build Script	13
3.5. File Deployment	14
4. Install BSP	15
4.1. Writing Boot Loaders.....	16
4.2. TFTP Server	17
4.3. NFS Server	18
4.4. Starting of U-Boot and Linux	19
5. APPENDIX	21

LIST OF FIGURES

FIGURE 1. LAYOUT	15
FIGURE 2. BOOT DIPSWITCH	15

i. VERSION LIST

Description	Version	Date	Author
Porting Guide	V0.0.1	2024/09/12	kevinlin

ii. HASHTAG

#V4H

#u-boot #kernel

#BSP

#ubuntu

1. Overview

Target :

Introduce how to download, compile and install RRC_Raptor files.

2. System Requirements

Reference PC software :

Distributor : Ubuntu
Description : Ubuntu 20.04.4 LTS
Release : 20.04
Codename : focal
Kernel : 5.15.0-43-generic

Reference PC hardware :

PC : Intel i9-11900K @ 2.50GHz
Width : 64 bits
Memory : 64G
Block device : 7.3TB Disk

3. Setup

Reference document :

Refer to <https://docs.yoctoproject.org/current/brief-yoctoprojectqs/index.html> for detail.

BB_VERSION	= 1.46.0
BUILD_SYS	= x86_64-linux
TARGET_SYS	= aarch64-poky-linux
MACHINE	= raptor
DISTOR	= poky
DISTOR_VERSION	= 3.1.11
SOC_FAMILY	= rcar:rcar-v4x:r8a779g0

META

meta-yocto-bsp	= tmp: 74b22db6879b388d700f61e08cb3f239cf940d18
meta-rcar-adas	= tmp: 89ca1415e3598789e9004363f9a38c1b3f41912d
meta-rtx	= v4h-raptor/meta-rtx:\${AUTOREV}
meta-networking	= tmp: 814eec96c2a29172da57a425a3609f8b6fcc6afe

3.1. Host Setup

Information :

When building on a machine running Ubuntu, the minimum hard disk space required is about 50 GB. It is recommended that at least 120 GB is provided, which is enough to compile all backends together.

The recommended minimum Ubuntu version is 20.04 or later.

The first download of source code and host setup must be executed when there is an Internet connection.

3.2. Host Packages

Essential Yocto Project host packages are :

```
$ sudo apt-get install gawk wget git-core diffstat unzip texinfo gcc-multilib  
build-essential chrpath socat cpio python python3 python3-pip python3-pexpect  
xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa  
libSDL1.2-dev pylint3 xterm libarchive-zip-perl rsync curl zstd p7zip-full libssl-dev
```

3.3. Yocto Project Setup

Git is set up properly with the commands below :

(this step may not be needed if the data has already been set)

```
$ git config --global user.name "Your Name"  
$ git config --global user.email "Your Email"  
$ git config --list
```

3.4. Source Download

Source Code :

```
$ mkdir RRC-Raptor-V4H  
$ cd RRC-Raptor-V4H  
$ git clone https://github.com/RetronixTechInc/rcar-bsp -b v4h-raptor
```

3.4.1. Using Build Script

Execute the build script :

Note: If you have executed Yocto build before, remove your build directory (\$WORK/build-*) so that future builds start from a clean environment.

Retronix provides a script for users to simplify their building flow.

```
$ cd rcar-bsp  
$ ./build_yocto_v3p28.sh
```

3.5. File Deployment

u-boot 、 Kernel & BSP file :

After successfully compiling, the binaries will be created in
[build-raptor-adas/tmp/deploy/images/raptor/] directory with the below
description :

Image	← kernel file
modules-raptor.tgz	← kernel driver
r8a779g0-raptor.dtb	← kernel device tree
rcar-image-adas-v4h-\${DATE}.rootfs.tar.bz2	← file system
u-boot.bin	← u-boot file

Each image build creates a U-Boot, a kernel, and an image type based on
the IMAGE_FSTYPES defined in the machine configuration file.

Most machine configurations provide a rootfs image (.tar).
The SD card image contains a partitioned image (with U-Boot, kernel, kernel
modules, rootfs, etc.) suitable for booting the corresponding hardware.

4. Install BSP

Layout :

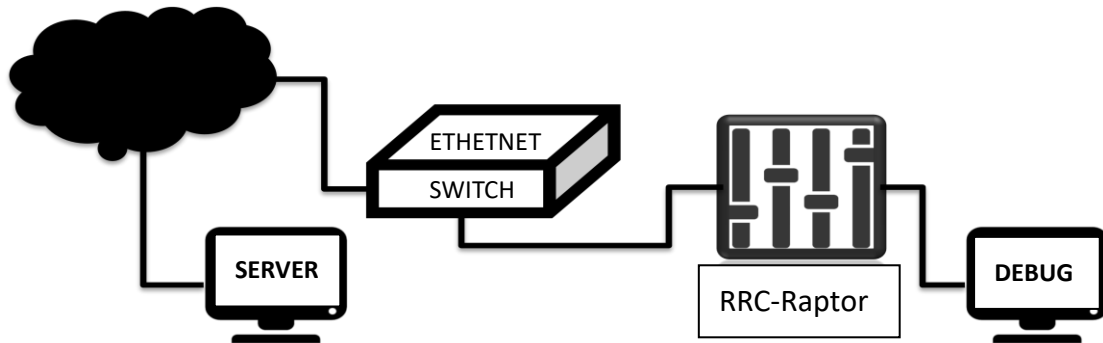


Figure 1. Layout

Raptor Kid DIP Switch :

SW26	SW35	SW31	SW27	SW30
1111 1111	1111 1111	1111 1111	1111 1111	1111 1111
SW28	SW36	SW33	SW29	SW32
0100 1011	0111 1110	1111 0100	1111 0111	1111 1111

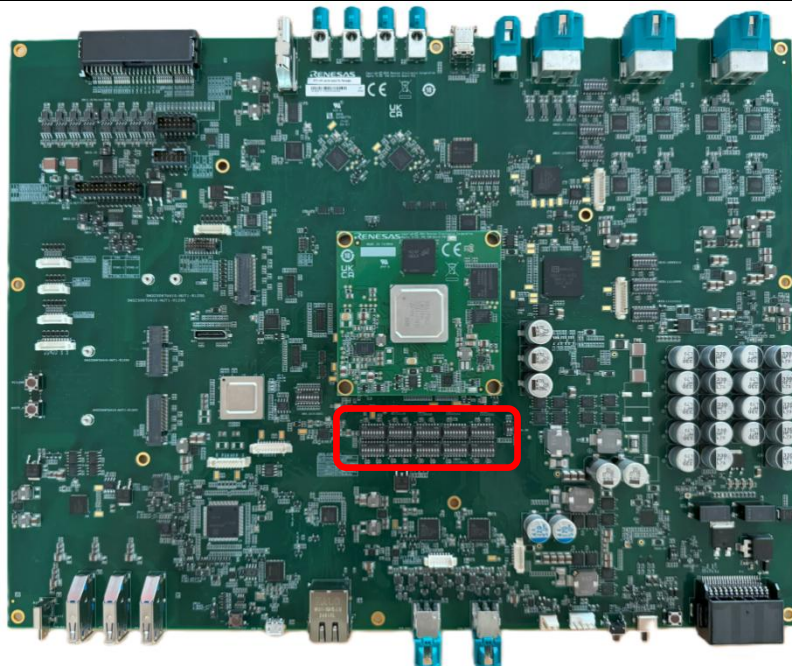


Figure 2. Boot dipswitch

4.1. Writing Boot Loaders

About “Writing boot loaders and u-boot”, Refer to SDK startup guide.

<< 102_RRC_Raptor_V4H_IPL_Porting_Guide.pdf>>

<< 201_RRC_Raptor_V4H_QSPI_Flash.pdf >>

4.2. TFTP Server

BSP space :

[Host PC server in ubuntu system]

Make sure you are connected to the internet. If there is any error, try apt update -y to update your application repository database.

SETP 1 : Install tftpd-hpa.

```
$ sudo apt install tftpd-hpa -y
```

STEP 2 : Change the following line to give R/W permission to TFTP folder.

```
$ sudo vi /etc/default/tftpd-hpa
```

```
# /etc/default/tftpd-hpa
```

```
TFTP_USERNAME="tftp"
```

```
TFTP_DIRECTORY="/var/lib/tftpboot"
```

```
TFTP_ADDRESS=":69"
```

```
#TFTP_OPTIONS="--secure"
```

```
TFTP_OPTIONS="--secure --create"
```

SETP 3 : Restart the tftpd-hpa service.

```
$ sudo service tftpd-hpa restart
```

4.3. NFS Server

KERNEL & DTS space :

[Host PC server in ubuntu system]

STEP 1 : Install the NFS kernel server package and rpcbind.

```
$ sudo apt install nfs-kernel-server  
$ sudo apt install rpcbind
```

STEP 2 : Configuring NFS Server.

```
$ sudo vi /etc/exports
```

```
/export/rfs *(rw,async,no_root_squash)
```

STEP 3 : Notify the NFS server after making any changes to the export file

```
$ exportfs -a
```

STEP 4 : Restart the NFS Service.

```
$ sudo service nfs-kernel-server restart
```

4.4. Starting of U-Boot and Linux

[Host PC server in ubuntu system]

STEP 1 : Prepare filesystem and kernel image and dtb.

Store Kernel image and device tree to TFTP root directory.

```
$ cp Image /var/lib/tftpboot  
$ cp r8a779g0-raptor.dtb /var/lib/tftpboot
```

Store filesystem rcar-image-adas-v4h-\${DATE}.rootfs.tar.bz2 to root directory of NFS server, and extract it as follows.

```
$ cp rcar-image-adas-v4h-${DATE}.rootfs.tar.bz2 /export/rfs  
$ cd /export/rfs  
$ tar xvf rcar-image-adas-v4h-${DATE}.rootfs.tar.bz2
```

[RRC-Raptor Device in Linux system]

STEP 2 : Set U-Boot environment variables.

The command indicates to mount the root file system through the **NFS server with read/write options**, the IP is **192.168.0.1**, the root directory is **/export/rfs**, and the **client IP is 192.168.0.20**. (IP value depends on domain settings.)

xx:xx:xx:xx:xx:xx is MAC address of raptor board.

```
=> setenv ethaddr xx:xx:xx:xx:xx:xx  
=> setenv ipaddr 192.168.0.20  
=> setenv serverip 192.168.0.1  
=> setenv loadaddr 0x48080000  
=> setenv loadaddr_dtb 0x48000000  
=> setenv tftp_kernel 'tftp ${loadaddr} Image'  
=> setenv tftp_dtb 'tftp ${loadaddr_dtb} r8a779g0-raptor.dtb'  
=> setenv bootcmd 'run bootargs;run tftp_kernel;run tftp_dtb;booti ${loadaddr} -  
${loadaddr_dtb}'
```

STEP 3 : Change the bootargs by U-Boot.

```
=> setenv bootargs 'setenv bootargs rw root=/dev/nfs  
nfsroot=${serverip}:/export/rfs,nfsvers=3 ip=dhcp cma=900M clk_ignore_unused  
consoleblank=0 video=DP-1:1920x1080-24@60'
```

STEP 4 : Save environment variables to persistent storage.

By the "saveenv" command, we do not need to execute Step 2 - Step 3 in next times.

```
=> saveenv
```

STEP 5 : Start Linux.

After board reset or execute the reset command, U-Boot is started. After countdown, Linux images are loaded, and boot messages are displayed.

```
=> reset
```

5. APPENDIX

RETRONIX Source Code Link :

<https://github.com/RetronixTechInc/rcar-bsp>